



Equipo markoboquinechoclojesu

Reporte de Práctica de Laboratorio — Programación Orientada a Objetos

EE: Temas Selectos de IIE II (Código: IEDI 18005)

Facultad de Instrumentación Electrónica — Universidad Veracruzana

Información General del Equipo

- **Número de Práctica:** Práctica 1
- **Título de la Práctica:** Sistemas de Control con Python y aplicación en ESP32
- **Fecha de Entrega:** 27/09/2026
- **Enlace al Repositorio de GitHub:** https://github.com/Mark0-9843/POO_markoboquinechoclojesu

Nombre Completo del Integrante	Matrícula UV	Usuario de GitHub
Boquiño Navarro Omar	S23013990	@lenoxzy
Ibarra Guzman Jose de Jesus	S23013916	@mookitagit
Morales González Marco Antonio	S23013998	@Mark0-9843
Salas Martinez Cesar David	S23013943	@Choclititito

Sección 1: Mapeo de Abstracción (Hardware → POO)

El proyecto tiene dos versiones. La Versión 1.0.0 es un simulador de reactor químico en Python con POO (lazo cerrado, modos manual/automático/pruebas e interlocks de seguridad). La Versión 2.0.0 lleva esos conceptos de control a hardware real: un ESP32 programado en C++ (entorno Arduino) con sensor DHT11, LDR, LED y un motor DC como ventilador.

1.1. Diagrama de Clases UML



1.2. Mapeo de Atributos (Características Físicas → Variables de Clase)

Versión 1.0.0 (simulador)

- **Clase Sensor (instancias: termómetro y presión):**
 - variable_fisica / unidad: magnitud medida y su unidad (Temperatura en °C; Presión en bar).
 - rango_min / rango_max: límites de operación del transductor según la especificación (0–150.0 °C y 0–15.0 bar).
 - sensibilidad: resolución mínima de la medición (0.01 en temperatura, 0.001 en presión).
 - decimales_medicion: cifras decimales con que se reporta la lectura (3 en temperatura, 2 en presión).
 - valor_actual: última lectura del muestreo analógico.
- **Clase Actuador:**
 - nombre: identifica el equipo físico (bomba, válvula).
 - estado: indica si el actuador está energizado (ON) o no (OFF).
- **Clase ActuadorProporcional (Bomba de Enfriamiento):**
 - rango_operacion_min / rango_operacion_max: límites de potencia de la bomba (0–100 %).
 - punto_operacion: porcentaje de operación aplicado (modulación proporcional).
- **Clase ActuadorDigital (Válvula de Alivio):**
 - punto_operacion: valor entero 0 (cerrada) o 1 (abierta), control ON/OFF.

Versión 2.0.0 (ESP32)

Variable / constante	Valor	Elemento físico que representa
LDR_PIN	GPIO 32	Entrada analógica (ADC) del divisor de voltaje con el LDR; mide la luz ambiental.
DHT_PIN	GPIO 4	Pin de datos del sensor DHT11 (temperatura y humedad).
LED_PIN	GPIO 19	Salida PWM hacia el LED.

Variable / constante	Valor	Elemento físico que representa
PWMA_PIN	GPIO 18	Salida PWM que controla la velocidad del motor DC a través del puente H.
AIN1_PIN / AIN2_PIN	GPIO 21 / 22	Definen el sentido de giro del motor en el puente H (fijo: AIN1 = HIGH, AIN2 = LOW).
STBY_PIN	GPIO 23	Habilita el driver del motor (standby en HIGH).
frecuencia / resolucion	5000 Hz / 8 bits	Parámetros PWM del periférico ledc: ciclo de trabajo de 0 a 255.
valorLDR	0–4095	Lectura cruda del ADC de 12 bits del LDR.
temperatura / humedad	float	Última lectura válida del DHT11 (-999 si hubo error de lectura).
brilloLED / velocidadMotor	0–255	Duty cycle actual aplicado al LED y al motor.
modoAutomatico / formatoJSON	bool	Estado del sistema: los sensores mandan o los comandos manuales mandan; formato de la telemetría.

1.3. Mapeo de Métodos (Acciones del Hardware → Funciones miembro)

Versión 1.0.0 (simulador)

- **Clase Sensor:**
 - `__init__(nom, var, min, max, sn, dec, und)`: configura el instrumento con sus características de hardware.
 - `leer_valor_actual()`: realiza el muestreo y devuelve la lectura en unidades de ingeniería.
 - `info()`: devuelve la ficha del sensor (rango, sensibilidad, decimales, lectura) que se muestra en el HMI.
- **Clases Actuador / ActuadorProporcional / ActuadorDigital:**
 - `encender()` y `apagar()`: cambian el estado del equipo (ON/OFF).
 - `ajustar(valor)`: fija el punto de operación validando el rango (0–100 % en la bomba; 0 o 1 en la válvula).
 - `info()`: reporta nombre, estado, punto de operación y rango del actuador.

Versión 2.0.0 (ESP32)

Función	Acción de hardware que ejecuta
<code>setup()</code>	Inicia el puerto serie a 115200 baudios, configura los pines del puente H (dirección fija y STBY activo), asocia los pines PWM con <code>ledcAttach()</code> e inicia el DHT11.
<code>procesarComandos()</code>	Lee comandos por serie sin bloquear el lazo: AUTO, MOTOR_ON/OFF, LED_ON/OFF, FORMAT_JSON/TEXT y STATUS.

Función	Acción de hardware que ejecuta
leerSensores()	Lee el LDR con <code>analogRead()</code> en cada ciclo y el DHT11 solo al llegar el intervalo de telemetría, validando con <code>isnan()</code> .
ejecutarLogicaAutomatica()	Lazo de control: el LED se regula de forma inversa al LDR (<code>map(valorLDR, 1000, 4095, 255, 0)</code>) y el motor se activa cuando la temperatura supera 30 °C.
actualizarActuadores()	Aplica el duty cycle a las salidas con <code>ledcWrite()</code> (LED y motor).
imprimirTelemetria()	Reporta el estado en texto o en JSON cada 2 s, usando <code>millis()</code> en lugar de <code>delay()</code> .

Sección 2: El Registro de Errores ("Bug Log")

#	Falla Física o Lógica Detectada	Causa Raíz (Hardware / Software)	Solución Técnica Implementada en Código / Conexión
1	El DHT11 devolvía lecturas inválidas (NaN) cuando se consultaba de forma continua.	El DHT11 necesita un intervalo mínimo entre lecturas y el <code>loop()</code> se ejecuta miles de veces por segundo.	El DHT11 se lee solo junto al temporizador de telemetría (2 s, con <code>millis()</code>). Si la lectura es inválida se conserva el código -999 y se imprime "ERR", y el ventilador no se activa con un dato erróneo.
2	El sistema respondía lento y la telemetría se retrasaba al esperar comandos por consola.	La lectura de cadenas por serie espera por defecto hasta 1 s y bloquea el lazo principal.	Se redujo el tiempo de espera con <code>Serial.setTimeout(10)</code> , se lee solo si hay datos (<code>Serial.available()</code>) y se sustituyó <code>delay()</code> por <code>millis()</code> . Además se limpian espacios y <code>\r</code> con <code>trim()</code> y se normaliza a mayúsculas.
3	Al configurar el PWM, las funciones <code>ledcSetup</code> y <code>ledcAttachPin</code> pedidas en la práctica no compilaban.	Con la versión 3.x del núcleo ESP32 para Arduino esas funciones fueron reemplazadas por una API unificada.	Se migró a <code>ledcAttach(pin, frecuencia, resolucion)</code> y se usa <code>ledcWrite(pin, duty)</code> directamente sobre el pin.
4	En el simulador, ajustar la bomba a 136.8 % o la válvula a un valor distinto de 0/1 dejaba al actuador en un estado inválido.	El ajuste no validaba el rango de operación del actuador.	El método <code>ajustar()</code> valida el rango y rechaza el valor con un mensaje de error ("fuera de rango" / "Es digital. Solo acepta valores de 0 o 1").

Sección 3: Justificación del Diseño Orientado a Objetos

1. Herencia y Reutilización de Código

Respuesta: Sí, en el simulador. La superclase es Actuador, que concentra el nombre y el estado (ON/OFF) comunes a cualquier equipo con métodos encender() y apagar(). De ella derivan las subclases ActuadorProporcional (bomba, 0–100 %) y ActuadorDigital (válvula, 0/1), que agregan su propio punto de operación y rango. Sensor se dejó como clase independiente porque una sola clase parametrizada por el constructor (rango, sensibilidad, decimales, unidad) cubre temperatura y presión sin necesidad de subclases. En la Versión 2.0.0 se optó por un diseño plano con funciones, por la sencillez del sistema (un sensor de cada tipo y dos salidas PWM).

2. Polimorfismo

Respuesta: ajustar(valor) e info() se comportan distinto según el objeto: en la bomba (ActuadorProporcional) acepta cualquier valor entre 0 y 100 % y lo rechaza fuera de rango; en la válvula (ActuadorDigital) solo acepta 0 o 1. Igualmente, ActuadorDigital redefine encender() y apagar() para fijar el valor 1 o 0. Así, el programa puede invocar el mismo método sobre cualquier actuador sin conocer su tipo.

3. Encapsulamiento y Ocultamiento de Datos

Respuesta: Todos los atributos de las clases son privados (por ejemplo estado, punto_operacion, rango_operacion_min/max, rango_min/max y valor_actual), de modo que ninguna parte del programa puede saltarse las validaciones. Se accede a ellos únicamente por métodos públicos: ajustar() valida el rango antes de modificar el punto de operación, leer_valor_actual() entrega la lectura del sensor e info() expone el estado. Con el interlock activo, las instrucciones del operario se ignoran. En el ESP32 las variables son globales y no están encapsuladas; una mejora futura sería agruparlas en clases equivalentes a las del simulador.

Sección 4: Evidencias de Funcionamiento (Validación Dual)

4.1. Captura de Pantalla de la Terminal / Monitor Serial

Versión 1.0.0 (Simulador): panel HMI en terminal con el estado de los objetos actualizándose en tiempo real. El interlock se activa al superar 85 °C o 12 bar.

Modo manual

```
=====
PANEL DE CONTROL INDUSTRIAL HMI - MODO: [MANUAL]
=====
[ACTUADORES]
+ [bomba ] Bomba de Enfriamiento | Estado: ON | Punto Op: 58.8% | Rango: [0.0% - 100.0%]
+ [válvula] Válvula de Alivio | Estado: OFF | Valor: 0 | Rango: [0 / 1 (Digital)]
=====
[SENSORES]
+ [termometro] Termometro | Var: Temperatura | Rango: [ 0.0 - 150.0 ] °C | Sensibilidad: 0.01 | Dec: 3 | Lectura: 0 °C
+ [presion ] Presion | Var: Presion | Rango: [ 0.0 - 15.0 ] Bar | Sensibilidad: 0.001 | Dec: 2 | Lectura: 1.75 Bar
=====
[REGISTRO DE EVENTOS EN VIVO (SCADA/HMI)]
[ 0 Bomba de Enfriamiento -> Punto de operación ajustado al 58.8%
[ 1 LECTURA] Presion: 1.75 Bar (Var: Presion)
[ 2 Válvula de Alivio -> Estado cambiado a: ENCENDIDO (ON)
[ 3 Válvula de Alivio -> Estado cambiado a: APAGADO (OFF)
=====
COMANDOS DISPONIBLES:
+ modo <manual/auto/pruebas> (Cambia el modo de operación)
+ encender <actuador> (Ej: encender bomba)
+ apagar <actuador> (Ej: apagar válvula)
+ ajustar <actuador> <val> (Ej: ajustar bomba 75.5 0 ajustar válvula 1)
+ leer <sensor> (Ej: leer termometro 0 leer presion)
+ terminar (Finaliza la simulación)
=====
Ingrese comando >> |
```

Cambio a modo auto

```
=====
PANEL DE CONTROL INDUSTRIAL HMI - MODO: [AUTO]
=====
[ACTUADORES]
+ [bomba ] Bomba de Enfriamiento | Estado: OFF | Punto Op:  0.0% | Rango: [0.0% - 100.0%]
+ [valvula] Válvula de Alivio      | Estado: OFF | Valor:  0 | Rango: [0 / 1 (Digital)]
=====
[SENSORES]
+ [termometro] Termometro          | Var: Temperatura | Rango: [ 0.0 - 150.0] °C | Sensibilidad: 0.01 | Dec: 3 | Lectura: 1.5 °C
+ [presion ] Presion              | Var: Presion     | Rango: [ 0.0 - 15.0] Bar | Sensibilidad: 0.001 | Dec: 2 | Lectura: 5.0 Bar
=====
[REGISTRO DE EVENTOS EN VIVO (SCADA/HMI)]
[SISTEMA] Modo de operación cambiado a: AUTO
[🔌] Bomba de Enfriamiento -> Punto de operación ajustado al 0.0%
[🔌] AUTO Bomba al 0.0% | ΔT: +1.50°C | Temp: 1.50°C
=====
COMANDOS DISPONIBLES:
+ modo <manual/auto/pruebas> (Cambia el modo de operación)
+ encender <actuador>        (Ej: encender bomba)
+ apagar <actuador>          (Ej: apagar válvula)
+ ajustar <actuador> <val>   (Ej: ajustar bomba 75.5 0  ajustar válvula 1)
+ leer <sensor>              (Ej: leer termometro 0  leer presion)
+ terminar                   (Finaliza la simulación)
=====
Ingrese comando >> |
```

Ciclos del modo auto

```
=====
PANEL DE CONTROL INDUSTRIAL HMI - MODO: [AUTO]
=====
[ACTUADORES]
+ [bomba ] Bomba de Enfriamiento | Estado: ON  | Punto Op: 13.5% | Rango: [0.0% - 100.0%]
+ [valvula] Válvula de Alivio     | Estado: OFF | Valor:  0 | Rango: [0 / 1 (Digital)]
=====
[SENSORES]
+ [termometro] Termometro          | Var: Temperatura | Rango: [ 0.0 - 150.0] °C | Sensibilidad: 0.01 | Dec: 3 | Lectura: 42.56 °C
+ [presion ] Presion              | Var: Presion     | Rango: [ 0.0 - 15.0] Bar | Sensibilidad: 0.001 | Dec: 2 | Lectura: 5.0 Bar
=====
[REGISTRO DE EVENTOS EN VIVO (SCADA/HMI)]
[🔌] AUTO Bomba al 9.6% | ΔT: +1.02°C | Temp: 40.81°C
[🔌] Bomba de Enfriamiento -> Punto de operación ajustado al 11.6%
[🔌] AUTO Bomba al 11.6% | ΔT: +0.92°C | Temp: 41.73°C
[🔌] Bomba de Enfriamiento -> Punto de operación ajustado al 13.5%
[🔌] AUTO Bomba al 13.5% | ΔT: +0.83°C | Temp: 42.56°C
=====
COMANDOS DISPONIBLES:
+ modo <manual/auto/pruebas> (Cambia el modo de operación)
+ encender <actuador>        (Ej: encender bomba)
+ apagar <actuador>          (Ej: apagar válvula)
+ ajustar <actuador> <val>   (Ej: ajustar bomba 75.5 0  ajustar válvula 1)
+ leer <sensor>              (Ej: leer termometro 0  leer presion)
+ terminar                   (Finaliza la simulación)
=====
Ingrese comando >> |
```

Modo pruebas: falla de temperatura inyectada, interlock de seguridad activado

```
=====
PANEL DE CONTROL INDUSTRIAL HMI - MODO: [PRUEBAS]
🚨 INTERLOCK DE SEGURIDAD ACTIVO - CONTROL DEL OPERARIO BLOQUEADO 🚨
=====
[ACTUADORES]
+ [bomba ] Bomba de Enfriamiento | Estado: ON  | Punto Op: 100.0% | Rango: [0.0% - 100.0%]
+ [valvula] Válvula de Alivio     | Estado: ON  | Valor:  1 | Rango: [0 / 1 (Digital)]
=====
[SENSORES]
+ [termometro] Termometro          | Var: Temperatura | Rango: [ 0.0 - 150.0] °C | Sensibilidad: 0.01 | Dec: 3 | Lectura: 160.526 °C
+ [presion ] Presion              | Var: Presion     | Rango: [ 0.0 - 15.0] Bar | Sensibilidad: 0.001 | Dec: 2 | Lectura: 5.0 Bar
=====
[REGISTRO DE EVENTOS EN VIVO (SCADA/HMI)]
[🔌] PRUEBA Monitoreo estable, sin anomalías inyectadas en este ciclo.
[🔌] ALERTA-PRUEBA Limite térmico excedido. Temp inyectada: 160.526 °C
[🔌] Bomba de Enfriamiento -> Punto de operación ajustado al 100.0%
[🔌] Válvula de Alivio -> Estado cambiado a: ENCENDIDO (ON)
[🔌] INTERLOCK Condición crítica (T=160.52°C, P=5.000Bar) -> Bomba forzada 100% / Válvula ABIERTA. Control del operario BLOQUEADO.
=====
COMANDOS DISPONIBLES:
+ modo <manual/auto/pruebas> (Cambia el modo de operación)
+ encender <actuador>        (Ej: encender bomba)
+ apagar <actuador>          (Ej: apagar válvula)
+ ajustar <actuador> <val>   (Ej: ajustar bomba 75.5 0  ajustar válvula 1)
+ leer <sensor>              (Ej: leer termometro 0  leer presion)
+ terminar                   (Finaliza la simulación)
🚨 Interlock de seguridad activo: comandos de control desactivados.
=====
Ingrese comando >>
```

Modo pruebas: presión excedida

```

=====
PANEL DE CONTROL INDUSTRIAL HMI - MODD: [PRUEBAS]
  INTERLOCK DE SEGURIDAD ACTIVO - CONTROL DEL OPERARIO BLOQUEADO
=====
[ACTUADORES]
+ [bomba ] Bomba de Enfriamiento | Estado: ON | Punto Op: 100.0% | Rango: [0.0% - 100.0%]
+ [valvula ] Válvula de Alivio | Estado: ON | Valor: 1 | Rango: [0 / 1 (Digital)]

[SENSORES]
+ [termometro] Termometro | Var: Temperatura | Rango: [0.0 - 150.0] °C | Sensibilidad: 0.01 | Dec: 3 | Lectura: 0 °C
+ [presion ] Presion | Var: Presion | Rango: [0.0 - 15.0] Bar | Sensibilidad: 0.001 | Dec: 2 | Lectura: 22.9 Bar

[REGISTRO DE EVENTOS EN VIVO (SCADA/HMI)]
[PRUEBA] Monitoreo estable. Sin anomalías inyectadas en este ciclo.
[ALERTA PRUEBA] Límite barométrico excedido. Presión inyectada: 22.9 Bar
[Bomba de Enfriamiento -> Punto de operación ajustado al 100.0%]
[Válvula de Alivio -> Estado cambiado a: ENCENDIDO (ON)]
[INTERLOCK] Condición crítica (T=0.00°C, P=22.90Bar) -> Bomba forzada 100% / Válvula ABIERTA. Control del operario BLOQUEADO.

COMANDOS DISPONIBLES:
+ modo <manual/auto/pruebas> (Cambia el modo de operación)
+ encender <actuador> (Ej: encender bomba)
+ apagar <actuador> (Ej: apagar válvula)
+ ajustar <actuador> <val> (Ej: ajustar bomba 75.5 0 ajustar válvula 1)
+ leer <sensor> (Ej: leer termometro 0 leer presion)
+ terminar (Finaliza la simulación)

Interlock de seguridad activo: comandos de control desactivados.
=====
Ingrese comando >>
  
```

Otras pruebas: valores fuera de rango rechazados por el actuador

```

=====
PANEL DE CONTROL INDUSTRIAL HMI - MODD: [PRUEBAS]
=====
[ACTUADORES]
+ [bomba ] Bomba de Enfriamiento | Estado: OFF | Punto Op: 0.0% | Rango: [0.0% - 100.0%]
+ [valvula ] Válvula de Alivio | Estado: OFF | Valor: 0 | Rango: [0 / 1 (Digital)]

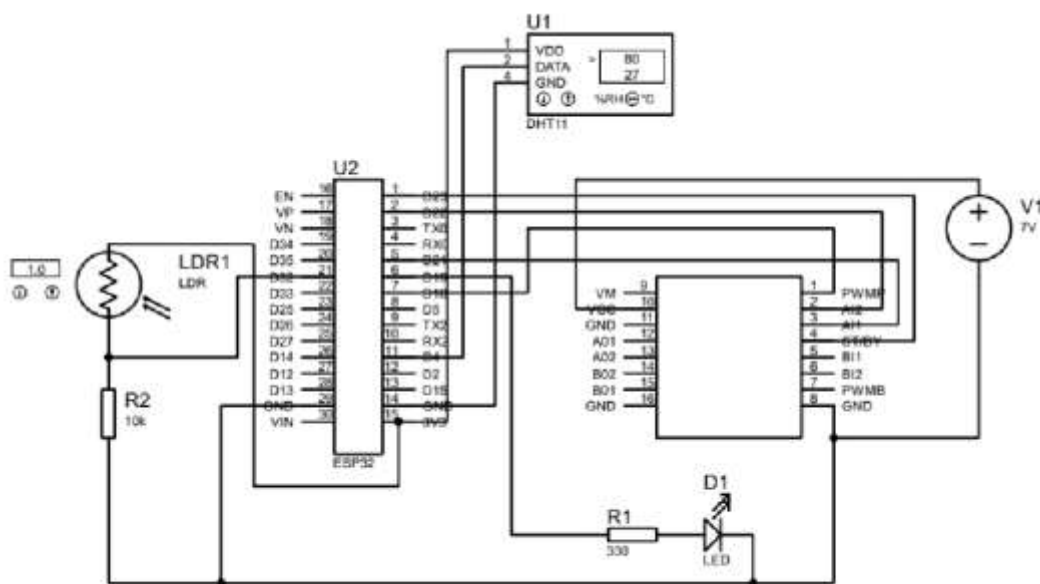
[SENSORES]
+ [termometro] Termometro | Var: Temperatura | Rango: [0.0 - 150.0] °C | Sensibilidad: 0.01 | Dec: 3 | Lectura: 0 °C
+ [presion ] Presion | Var: Presion | Rango: [0.0 - 15.0] Bar | Sensibilidad: 0.001 | Dec: 2 | Lectura: 5.0 Bar

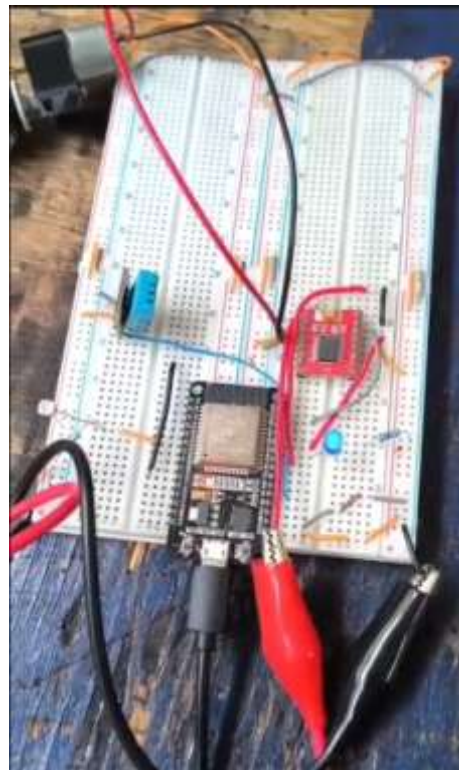
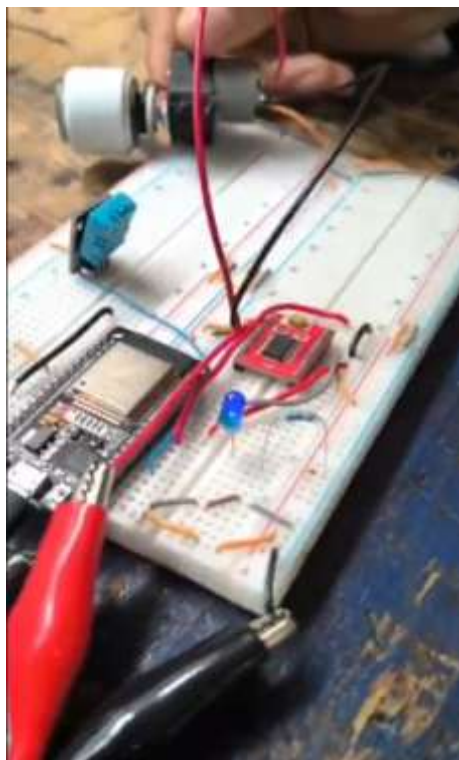
[REGISTRO DE EVENTOS EN VIVO (SCADA/HMI)]
[SISTEMA] Modo de operación cambiado a: PRUEBAS
[PRUEBA] Inyectando fallo: Intentando modular bomba fuera de limite...
[ERROR] Bomba de Enfriamiento -> Valor 130.0% fuera de rango (0% - 100%).
[PRUEBA] Inyectando fallo: Valor lógico no reconocido en válvula...
[ERROR] Válvula de Alivio -> Es digital. Solo acepta valores de 0 o 1.

COMANDOS DISPONIBLES:
+ modo <manual/auto/pruebas> (Cambia el modo de operación)
+ encender <actuador> (Ej: encender bomba)
+ apagar <actuador> (Ej: apagar válvula)
+ ajustar <actuador> <val> (Ej: ajustar bomba 75.5 0 ajustar válvula 1)
+ leer <sensor> (Ej: leer termometro 0 leer presion)
+ terminar (Finaliza la simulación)

=====
Ingrese comando >>
  
```

4.2. Fotografía del Circuito Físico Armado





```

1 // Definición de pines de salida
2 #define PIN_LED 15
3 #define PIN_MOTOR 12
4 #define PIN_TEMP 34
5 #define PIN_LDR 35
6
7 // Definición de constantes
8 const int LED_ON = 1;
9 const int LED_OFF = 0;
10 const int MOTOR_ON = 1;
11 const int MOTOR_OFF = 0;
12
13 // Definición de variables
14 int led_state = LED_OFF;
15 int motor_state = MOTOR_OFF;
16 float temperature = 0;
17 float ldr_value = 0;
18
19 // Función para encender/apagar el LED
20 void control_led() {
21   digitalWrite(LED_PIN, led_state);
22 }
23
24 // Función para encender/apagar el motor
25 void control_motor() {
26   digitalWrite(MOTOR_PIN, motor_state);
27 }
28
29 // Función para leer la temperatura
30 void read_temperature() {
31   temperature = readTemp();
32 }
33
34 // Función para leer el valor del LDR
35 void read_ldr() {
36   ldr_value = readLDR();
37 }
38
39 // Configuración de pines de salida
40 void setup() {
41   pinMode(LED_PIN, OUTPUT);
42   pinMode(MOTOR_PIN, OUTPUT);
43   pinMode(TEMP_PIN, INPUT);
44   pinMode(LDR_PIN, INPUT);
45 }
46
47 // Configuración de velocidad del motor
48 void set_motor_speed(int speed) {
49   // Código para configurar la velocidad del motor
50 }
51
52 // Función principal
53 void loop() {
54   // Leer temperatura y LDR
55   read_temperature();
56   read_ldr();
57
58   // Control de lógica
59   if (temperature > 30) {
60     led_state = LED_ON;
61     motor_state = MOTOR_ON;
62   } else if (ldr_value > 500) {
63     led_state = LED_ON;
64     motor_state = MOTOR_ON;
65   } else {
66     led_state = LED_OFF;
67     motor_state = MOTOR_OFF;
68   }
69
70   // Control de LED y motor
71   control_led();
72   control_motor();
73
74   // Retraso
75   delay(1000);
76 }

```

Versión 2.0.0 (ESP32): ESP32, DHT11, LDR, driver del motor, motor DC y LED sobre protoboard.

4.3. Enlace Directo al Video Demostrativo (Máximo 1.5 minutos)

- **Versión 1.0.0 (Simulador):** <https://youtu.be/Q5a-OSLc4h8>
- **Versión 2.0.0 (ESP32):** [https://youtube.com/shorts/ bv4KVSH0VE](https://youtube.com/shorts/bv4KVSH0VE)